

Registrando alterações: **status**, **add** e **commit**

O Git não salva alterações automaticamente. Nós decidimos quando criar uma nova versão do projeto, usando `status`, `add` e `commit`.

 O CICLO BÁSICO

Do arquivo ao histórico



Working Directory

Onde você faz as alterações

`git add`



Staging Area

Área de preparação

`git commit`



Histórico

Versão salva



Um **commit** é como uma **fotografia** do projeto naquele momento — e o `git add` é a preparação antes da foto.

Qual é a situação do projeto?

   antes de qualquer alteração

```
$ git status  
On branch main  
No commits yet  
nothing to commit
```

   depois de editar o README

```
$ git status  
Changes not staged for commit  
# o Git percebeu a alteração, mas ela ainda não foi preparada
```



git status responde: "Qual é a situação atual do meu projeto?"

 GIT ADD

Preparando as alterações

 preparar um arquivo

```
$ git add README.md
```

```
$ git status
```

```
Changes to be committed
```

```
# o arquivo mudou de seção: agora está pronto para virar uma versão
```



`git add .` prepara **todos** os arquivos modificados de uma vez.

 GIT COMMIT

Registrando a versão

 criar o commit

```
$ git commit -m "Adiciona descrição inicial ao projeto"
# -m é abreviação de --message (a mensagem do commit vem logo em seguida)
$ git status
nothing to commit, working tree clean
```



Cada commit representa **um ponto da evolução** do projeto.

Uma boa mensagem responde: o que mudou?

❌ EVITE

```
teste  
alterações  
123
```



✅ PREFIRA

```
Adiciona descrição inicial  
Cria página inicial  
Atualiza arquivo README
```

Daqui a seis meses, você ainda vai lembrar o que mudou?

✦ BOAS PRÁTICAS

Commitando bem



Consulte o status sempre

`git status` é o comando que você mais vai usar.



Faça commits pequenos

Evite alterar dezenas de arquivos antes de criar um commit.



Mensagens objetivas

"Adiciona formulário de contato" em vez de "Atualizações".

 PRÁTICA GUIADA

O ciclo completo

✓ Adicionar uma linha ao `README.md` .

✓ Rodar `git status` .

✓ Rodar `git add README.md` e conferir o status.

✓ Criar o commit e ver `working tree clean` .

ENCERRAMENTO

Primeira versão registrada!

O processo acontece em três etapas: verificar as alterações, preparar os arquivos e criar um commit.



Você sabia?

Um projeto profissional pode ter milhares de commits ao longo da vida. Em grandes projetos open source, como o Linux ou o VS Code, o histórico registra anos de evolução — por isso boas mensagens de commit são tão valorizadas.



Próxima aula: consultar o **histórico** e boas práticas de commit com `git log`.